

Written Report

Date: 06/28/2022

Word count:987

By submitting this work, I declare that this is my original work that I wrote individually and that any instances of academic impropriety could result in this work receiving an automatic fail.

ALS assignment

Abstract

Python coding is easy to use and can be employed to create an arcade game. The group project focuses on the methods and functions of the Python coding language integrated into designing an arcade game. The programming of our design is based on OOP to make coding more efficient for a simple 2-D shooting game. This report also focuses on OOP methods associated with specific classes or functions, which have advantages for coding.

Introduction

Python coding allows Functional Programming (FP) and Object-Oriented Programming (OOP) and implements to experience the interpreted language (McGugan, 2007). The purpose of the eight-member project team is to design an arcade game by applying knowledge, understanding of the Python language, and implementing methods and functions using PyGame libraries. The project aims to use the methods and functions of Python to create an arcade game that has to have projectile action, characters, background, and a score counter. This report will first focus on the aspects of the Python language used in the arcade game design and then show the details of the features that make designing the game possible. Next, it illustrates the differences between methods and functions. Finally, it examines the differences between OOP and FP and why OOP is used in the game more.

As mentioned above, the goal is to come to a state where we fully understand and use Python functions, objects, and classes to create an arcade game. This report suggests OOP makes class states more manageable by dividing them into simple objects that make the program more understandable, easy to use, read and debug. FP follows an entirely different method that does not has a state, and the goal is not to cut the state but to manage it better.

Literature Review

Python makes coding easier because it shortens development cycles and increases productivity for game design and development (Chen & Liu, 2022). It has Sprite and Surface in the PyGame package, which has all features needed to construct an in-game environment (Sahni et al., 2015). In addition, PyGame's core is compact, and maintenance is manageable, allowing functions to be created independently as external libraries (Koupritzioti & Xinogalos, 2022).

Methods are functions that are created within a class under which they operate. Functions, however, do not demand any classes to be defined (McGugan, 2007). Methods are connected with objects, while functions are not. Method's first argument needs to have 'self,' but functions can be written without 'self' and have more than zero arguments (Kelly, 2019). Also, while it is not possible to call methods on its name, it is possible to call functions on its name. They have invoked objects and the dot operator to do it. It is the subject of hundreds of distinct functions (Craven, 2016). For that reason, methods are preferred more in making an arcade game.

An arcade game can be programmed with two main methods. These are called FP and OOP. FP helps construct software by focusing on the functions and mathematical operations rather than executions (Kelly, 2019). It simply gathers functions and makes calculations. OOP focuses on objects to describe everything to the program. These objects can contain data, code, and attributes and can be changed with the help of methods or functions (McGugan, 2007). The advantage of using OOP is that it is easy to make code changes since the encapsulation of functions allows to change the whole class (Kelly, 2019). In addition, readability and understanding of the code are much more manageable.

Methodology

The group project initially started by importing PyGame libraries. Then, the general game design was discussed, and the ideas were added to the Miro platform, a collaborative whiteboard planning the workflows. Finally, the code was written collectively in the platform Replit. Firstly, the main projectile action was coded and defined as player and bullet. In Figure 1, the player and the bullet were coded as objects because these templates are in classes. Also, the attributes of these objects can be changed if necessary. For example, the color of the bullet can be changed from "WHITE" to "GRAY" by using the method of `image.fill`. As shown in Figure 2, the player template is a green square, and the projectile is a white rectangle. It seems simple, but it fulfills the basics of a projectile action.

First, all game images, such as the background and models, were added to the game using functions. Then, the images were defined as Portable Network Graphics (png) in Figure 3. Next, the in-game text was put together, like the wave, and the enemy count is shown in Figure 4; these texts were added by using functions (user-defined functions). After that, the game menu was designed the same way defined in the in-game texts shown in Figures 5 and 6. Lastly, a mode called "Polska" was added to the game. Polska mode is activated when the player gets the herb, as seen in Figure 7, and gains extra health and frequent projectile action. This mode lasts for 30 seconds. Polska mode is shown in Figure 8.

Results

The basic programming of the Python-based 2D arcade game is completed. The game includes projectile, movement, shooting, and score modules. The OOP method is used rather than FP because the arcade game is based on objects. Methods are used more than functions of the project because classes are used to define to create an object. Overall, a workable product is achieved.

Discussion

The goal was to create an arcade game using a Python library to show the differences between Python's methods and functions. For programming, OOP and FP are options to use. Nevertheless, FP has an entirely different approach which eliminates the states in a design pattern. However, OOP makes states more manageable by breaking them down into small objects that make the program clear. Therefore, OOP is used in 2-D game design because methods branch into classes. Although researchers argue that FP prevents data from changing in favor of setting new values during data calculations, which is beneficial for mathematical operations (Kelly, 2019), in our project, OOP is preferred.

This project could not control all variables, as one of the limitations of Python's slow runtime is a result of doing extra work while running. Another limitation is execution errors, such as the possibility of an integer number variable carrying a string, resulting in runtime errors.

Conclusion

The project involves the development of the arcade game on the Python coding of the PyGame library and using OOP to code objects by using the class method as OOP makes the coding more understandable and object-oriented. Future studies should include more variables because of the limitations of the execution of coding time and errors.

Bibliography/Reference list

Chen,X. and Liu, K. (2022) ‘The Design of Airplane War Game Based on Python’, Journal of Simulation, 10(1).Available at:
[http://www.journalofsimulation.com/d/file/previous/2022%20Volume%201/Vol%2010,%20No%201%20\(2022\)/2022-03-03/dca88ee1d2bb021a9be97912342383bb.pdf](http://www.journalofsimulation.com/d/file/previous/2022%20Volume%201/Vol%2010,%20No%201%20(2022)/2022-03-03/dca88ee1d2bb021a9be97912342383bb.pdf) (Accessed:2 June 2022)

Craven, P. (2016) Program Arcade Games. ISBN-13. New York: Apress Media, pp. 1–76.

Kelly, S. (2016) Python, PyGame and Rasberry Pi Game Development. ISBN-13. New York: Apress Media, pp. 1–10.

Koupritzioti,D. and Xinogalos, S. (2019). ‘PyDiophantus maze game: Play it to learn mathematics or implement it to learn game programming in Python’ Education and Information Technologies, 25(4), 2747–2764. <https://doi.org/10.1007/s10639-019-10087-1>

McGugan, W. (2007) Beginning Game Development With Phyton and Pygame. ISBN-13. New York: Apress Media, pp. 1–54.

Sahni, N., Srinivasan, K., & Savla, H. (2015). ‘A review on developing an arcade game machine and an arcade game using Raspberry Pi and pygame’, International Journal of Computer Applications, 120(17), 37-39. doi:10.5120/21323-4353 (Accessed:2 June 2022)

Appendices

Figure 1

```

17 # defining what player is
18 class Player(pygame.sprite.Sprite):
19     def __init__(self):
20         pygame.sprite.Sprite.__init__(self)
21         self.image = pygame.Surface((50, 40))_# setting a rectangle
22         self.image.fill(GREEN)
23         self.rect = self.image.get_rect()
24         self.rect.centerx = WIDTH / 2_# defining center
25         self.rect.bottom = HEIGHT - 10
26         self.speedx = 0_# the initial speed on x and y
27         self.speedy = 0_# basically defining the variables
28     def update(self):
29         self.speedx = 0_# every iteration the speed is set to zero
30         self.speedy = 0_# so that it moves only when button is pressed
31         keystate = pygame.key.get_pressed()_# setting of controls
32         if keystate[pygame.K_a]:
33             self.image = pygame.Surface((40, 50))
34             self.image.fill(GREEN)
35             self.speedx = -3
36             self.speedy = 0
37         elif keystate[pygame.K_d]:
38             self.image = pygame.Surface((40, 50))
39             self.image.fill(GREEN)
40             self.speedx = 3
41
42
43
44 class Bullet(pygame.sprite.Sprite):
45     def __init__(self, x, y):
46         pygame.sprite.Sprite.__init__(self)
47         self.image = pygame.Surface((5, 10))
48         self.image.fill(WHITE)
49         self.rect = self.image.get_rect()
50         self.rect.bottom = y + 40
51         self.rect.centerx = x
52         self.speedy = 4
53         self.speedx = 0
54     def update(self):
55         self.rect.y += self.speedy
56         self.rect.x += self.speedx
57         if (self.rect.bottom < 0) or (self.rect.top > HEIGHT) or (self.rect.left > WIDTH) or (self.rect.right < 0):
58             self.kill()

```

Figure 2

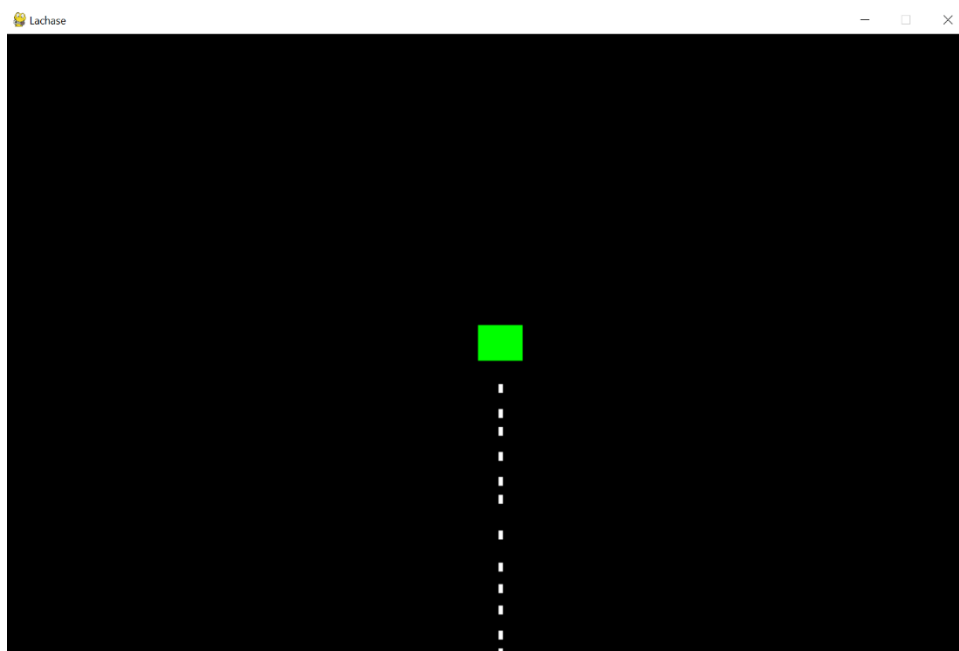


Figure 3

```

7
8 WIDTH = 1080
9 HEIGHT = 700
10 FPS = 60
11
12 img_dir = path.join(path.dirname(__file__), 'img')
13 background1 = pygame.image.load(path.join(img_dir, 'lvl1_background.png'))
14 background_rect1 = background1.get_rect()
15 background2 = pygame.image.load(path.join(img_dir, 'lvl2_background.png'))
16 background_rect2 = background2.get_rect()
17 background3 = pygame.image.load(path.join(img_dir, 'lvl3_background.png'))
18 background_rect3 = background3.get_rect()
19 mob_img = pygame.image.load(path.join(img_dir, 'mob.png'))
20 background_rect = []
21 farm_img1 = pygame.image.load(path.join(img_dir, 'base1.png'))
22 farm_img_rect1 = farm_img1.get_rect()
23 farm_img2 = pygame.image.load(path.join(img_dir, 'base2.png'))
24 farm_img_rect2 = farm_img2.get_rect()
25 farm_img3 = pygame.image.load(path.join(img_dir, 'base3.png'))
26 farm_img_rect3 = farm_img3.get_rect()
27 special_effect = pygame.image.load(path.join(img_dir, 'special_effect.png'))
28 r_cow = pygame.image.load(path.join(img_dir, 'r_cow.png'))
29 polska_cow = pygame.image.load(path.join(img_dir, 'polska_cow.png'))

```

Figure 4

```

font_name = pygame.font.match_font('arial') #text

def draw_text(surf, text, size, x, y, color):
    font = pygame.font.Font(font_name, size)
    text_surface = font.render(text, True, color)
    text_rect = text_surface.get_rect()
    text_rect.midtop = (x, y)
    surf.blit(text_surface, text_rect)

def draw_shield_bar(surf, x, y, pct, color, n):
    if pct < 0:
        pct = 0
    BAR_LENGTH = 100
    BAR_HEIGHT = 10
    fill = (pct / 100) * BAR_LENGTH * n
    outline_rect = pygame.Rect(x, y, BAR_LENGTH, BAR_HEIGHT)
    fill_rect = pygame.Rect(x, y, fill, BAR_HEIGHT)
    pygame.draw.rect(surf, color, fill_rect)
    pygame.draw.rect(surf, WHITE, outline_rect, 2)

```

Wave 2
enemies left: 3

Figure 5

```

1 import pygame
2 import sys
3 from os import path
4
5 pygame.init()
6
7 WIDTH = 1080
8 HEIGHT = 700
9 spawn = True
10 img_dir = path.join(path.dirname(__file__), 'img')
11 Play_Button = pygame.image.load(path.join(img_dir, 'Start.png'))
12 Settings_Button = pygame.image.load(path.join(img_dir, 'Settings.png'))
13 Exit_Button = pygame.image.load(path.join(img_dir, 'Exit.png'))
14
15 i = [Play_Button, Settings_Button, Exit_Button]
16
17 pygame.display.set_caption('LACHASE')
18 icon = pygame.image.load(path.join(img_dir, 'cow.png'))
19 pygame.display.set_icon(icon)
20 Screen = pygame.display.set_mode((1080, 700), pygame.RESIZABLE, 64)
21 BG = pygame.image.load(path.join(img_dir, 'Group16.png'))
22 BG_rect = BG.get_rect()
23 all_sprites = pygame.sprite.Group()
24

```

Figure 6

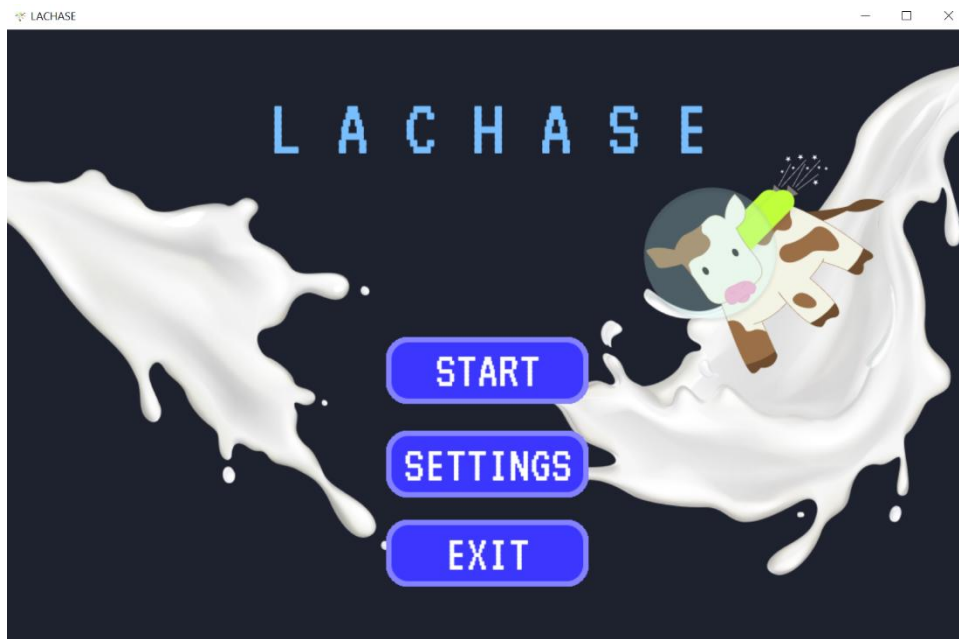


Figure 7

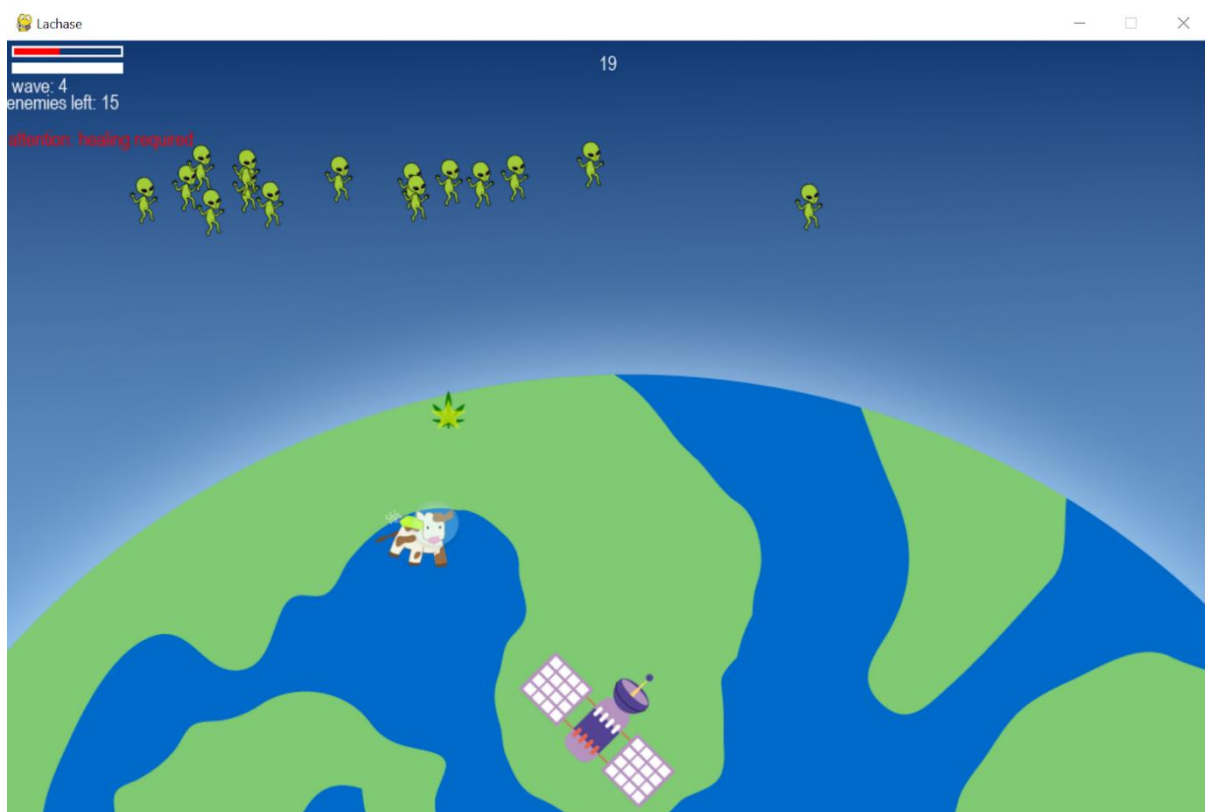


Figure 8

